Article Title

Saksham Diwan[1], Harshaan Chugh[1], Shuo Cao[2], Erik Garrison[2*], Giulia Guidi[1,3*]

[1*]Department of Computer Science, Cornell University, 107 Hoy Rd, Ithaca, 14853, New York, USA.

[2*]Department of Genetics, Genomics and Informatics, University of Tennessee Health Science Center, Memphis, 38163, Tennessee, USA.

[3]Applied Mathematics and Computational Research Division, Lawrence Berkeley National Laboratory, 1 Cyclotron Road, Berkeley, 94720, California, USA.

*Corresponding author(s). E-mail(s): egarris5@uthsc.edu; gg434@cornell.edu;
Contributing authors: sd966@cornell.edu; hsc53@cornell.edu;

Abstract

Motivation: Self-aligning a pangenome of $\boldsymbol{H}$ haplotypes with FastGA finds matches across all haplotype pairs in a single scan of one sorted $\boldsymbol{k}$-mer table, so the scan grows as $\boldsymbol{O(H \cdot L)}$ rather than the $\boldsymbol{O(H^2 \cdot L)}$ of explicit all-pairs alignment [1, 2]. The table itself, however, grows with $\boldsymbol{H \cdot L}$ and soon exceeds the memory of a single node. Distributing it across nodes removes that limit, but the result is only useful if it reproduces FastGA's alignments exactly.
Results: [TODO: results summary, after the primate and human experiments (shared)]
Availability and Implementation: DistributedPanGA is implemented in C with MPI and is available at https://github.com/SakshamDiwan/DistributedPanGA. Benchmark pangenomes are from Zenodo [3]. [TODO: license and archived release]
Contact: egarris5@uthsc.edu, gg434@cornell.edu
Supplementary Information: Supplementary data are available at *Bioinformatics* online.

1 Introduction

[TODO: Introduction (Saksham)]

1.1 Motivation

[TODO: Saksham]

1.2 Problem

[TODO: Saksham]

1.3 Prior work

[TODO: Saksham]

1.4 Contributions

[TODO: Saksham]

2 System and methods

2.1 FastGA

FastGA [1] aligns genome assemblies on a single shared-memory node. It stores the input as a genome database (GDB) of 2-bit packed sequence and builds a genome index (GIX): the sorted 40-mers sampled at closed-syncmer positions (Section 3.1), with the longest common prefix (LCP) of each entry with its predecessor and every occurrence's contig, offset and strand. Seeds are adaptive seeds (adaptamers): for each occurrence, the longest prefix it shares with another indexed k-mer, kept only if fewer than f entries share it ($f = 10$ by default), and paired with every other occurrence of that prefix. Seeds are sorted by contig pair and diagonal, diagonal bands with sufficient seed coverage are extended into local alignments, and `ALNtoPAF` converts FastGA's `.1aln` output to PAF. For a pangenome, all haplotypes are placed in one GDB and aligned against themselves, so one index and one merge cover every pair of haplotypes [2].

For H haplotypes of length about L, indexing and the merge scan are $O(HL)$. So is seed generation, with at most $f - 1$ seeds per occurrence, but only because adaptamers occurring f or more times (e.g. a 40-mer conserved in f haplotypes) are discarded. Alignment has no such bound: when most haplotype pairs align end to end, the output alone grows quadratically in H.

2.2 DistributedPanGA

DistributedPanGA [4] computes the same self-alignment as FastGA with P single-threaded MPI ranks, which read one shared GDB on demand. The pipeline has six stages (Section 3).

1. **Extraction:** each rank emits one record per sampled k-mer occurrence in its range of whole contigs.
2. **Distributed sort:** records are exchanged by k-mer prefix and sorted locally; the slices, in rank order, hold FastGA's index entries in k-mer order.
3. **Seed generation:** each rank runs FastGA's adaptamer merge on its slice.
4. **Seed redistribution:** each seed goes to the single rank that owns its contig pair.
5. **Alignment:** each rank aligns its own pairs with FastGA's code.
6. **Output:** each rank writes PAF with `ALNtoPAF`; the union of the per-rank files is the result.

Syncmer selection, the adaptamer merge, alignment and PAF conversion are FastGA code, kept close to upstream. Ours are the extraction partition and in-window k-mer packing, the distributed sort, an adapter that presents a rank's slice through FastGA's index interface in place of a GIX file, the deterministic assignment of contig pairs to ranks, and the two bounded all-to-all exchanges (Section 3.2).

Our correctness criterion is exact agreement with an independent single-node FastGA run on the same GDB: both PAF files are sorted bytewise, keeping duplicates, and compared byte for byte. This multiset equality is stronger than matching alignment counts.

3 Algorithm

3.1 Seed extraction

Extraction reproduces FastGA's k-mer sampling exactly. Each rank scans a contiguous range of whole contigs, balanced by length, in windows of up to 10 Mbp. The start j of a 12-mer is selected when the 12-mer is a closed syncmer: the minimum of its five canonical 8-mers, under FastGA's fixed (non-lexicographic) order, occurs at its first or last 8-mer position. With ties, an occurrence at either end suffices.

Each selected position yields up to two records: the forward 40-mer starting at j and the reverse complement of the 40-mer ending at the last base of the syncmer; a strand is skipped where its 40-mer would leave the contig. A record stores the 2-bit packed k-mer with its contig, position and strand. Occurrences, not k-mer counts, are needed because each seed pairs two occurrences.

Every window after the first is extended 40 bases to the left, so that reverse-complement k-mers and forward k-mers starting before the window lie inside it. Only a forward k-mer that runs past the right edge of a non-final window is read separately from the GDB. The scan state carries across windows, so the selected positions and the order of emitted records do not depend on the window size.

3.2 Data movement between ranks

Ranks exchange data twice: records during the distributed sort, and seeds before alignment.

Distributed sort.

Each record is assigned to one of 1,024 buckets by the first five bases of its k-mer. One all-reduce of bucket counts lets every rank compute the same split into P contiguous bucket ranges of about $1/P$ of the records, by the rule FastGA uses to divide its index among threads. Each rank then radix-sorts its slice and recomputes LCPs, correcting the first against the preceding rank's last k-mer. Because buckets are never split, seed generation needs no communication: FastGA's merge pairs only occurrences sharing their first 12 bases, so every candidate group, including the full count tested against the cutoff, lies on one rank, and each rank produces exactly its share of FastGA's seeds.

Seed redistribution.

Each seed is sent to the rank that owns its contig pair. Every rank computes the same pair-to-rank map from contig lengths alone, giving pairs in decreasing order of length product to the rank with the smallest accumulated product, so no communication is needed. The merge writes each seed directly into an in-memory buffer for its destination rank.

Bounded exchanges.

Both exchanges are personalized all-to-all transfers of fixed-size records whose volume per rank can exceed the 32-bit counts and displacements of `MPI_Alltoallv` ($2^{31} - 1$ bytes). Sizes are therefore exchanged as 64-bit integers, and a transfer that exceeds a per-round budget is split into rounds of `MPI_Alltoallv`. Each round moves at most a fixed number of whole records per peer, and every received slice is written at its final offset, so the result is byte-identical to a single unbounded exchange for any number of rounds. The budget bounds only staging memory: the complete send and receive data stay resident (during the sort, a rank holds its extracted records, their packed send copy and its received slice), so peak memory per rank is set by its volume of records and seeds, not by the budget (Section 4).

3.3 Distributed alignment

[TODO: Saksham: contig-pair assignment, seed routing, per-rank alignment]

4 Implementation

4.1 Extraction I/O

Per-rank timers showed that extraction dominated early runs and barely shortened as ranks were added. Although the scan held its window in memory, each selected k-mer was fetched again from the GDB. The bytes came from the file buffer, but every fetch repositioned the file: system-call tracing counted one `lseek` per record. Kernel time per rank stayed roughly constant as work per rank fell. We did not isolate why (shared-file-system contention is one candidate); the change removes the calls regardless.

Each k-mer is now packed from the window buffer, extended by 40 bases of left context (Section 3.1); only forward k-mers crossing a window's right edge are still fetched. Both paths share one packing routine and the selection loop is unchanged, so only the source of the bases changes, not which records are emitted or in what order.

Validation has three levels. Extraction output is byte-identical to the previous implementation and to a record dump from an instrumented copy of FastGA's `GIXmake`. Unit tests with a 64-base window compare against a single-window run, check that both paths fire and together account for every record, and run under address and undefined-behaviour sanitizers. End-to-end runs match the FastGA reference (Section 2.2).

4.2 Large-count exchanges

[TODO: Saksham: O2 implementation]

4.3 In-memory seeds

[TODO: Saksham: O4 implementation]

5 Results

5.1 Experimental setup

[TODO: owner to be decided: Perlmutter nodes, build, rank placement, FastGA reference version, datasets [3]]

5.2 Yeast and primate datasets

[TODO: Saksham]

5.3 Human chromosome 6

[TODO: Harshaan, after the human runs]

5.4 Scaling

[TODO: Harshaan and Saksham, after the human runs]

6 Discussion

6.1 Small-dataset results

[TODO: Saksham: interpretation of the yeast and primate results]

6.2 Scaling bottlenecks

[TODO: Harshaan and Saksham, after the human runs]

6.3 Human-scale validation

[TODO: Harshaan, after the human runs (index 61244453, FastGA reference 61244455, Distributed-PanGA 61244457)]

6.4 Limitations and future work

Alignment is now the dominant stage, and its cost is unevenly distributed across ranks. Contig pairs are assigned statically by length product, whereas a task's time depends on how much sequence it aligns; in primates4 the costliest tasks paired short, repeat-rich contigs with few seeds but very long alignments. Task granularity is a harder limit: each pair is aligned by one rank, so the largest task bounds alignment time from below. In primates14 (earlier revision), that task was the self-alignment of the longest contig, and total alignment work was only about 14 times its cost, capping alignment speedup near 14× at any rank count.

Workload-aware load balancing.

Contig pairs could be weighted by estimated work rather than length, as in the multiway number partitioning of ELBA [5], provided every rank still derives the assignment without communication. However, pre-alignment features such as seed counts only partly predict alignment time in our measurements. Replaying measured task times under a dynamic schedule (self-alignments first, by contig length) shortens the slowest rank in simulation only; it is not implemented, and it conflicts with sending seeds to their owner before alignment. No schedule beats the largest-task bound; that requires splitting self-alignments across ranks while reproducing FastGA's alignments exactly, an open problem.

GPU acceleration.

Syncmer selection and radix sorting are regular, data-parallel passes over packed 2-bit sequence and candidates for GPU offload, but they now take a small share of run time; alignment, which dominates, is irregular and harder to offload.

Other limits.

Ranks are single-threaded, so cores are used only as additional ranks. The contig-pair map stores one byte per ordered pair on every rank, which caps the rank count at 127 and grows quadratically with the number of contigs, a constraint for fragmented assemblies. Work is partitioned at whole contigs for extraction and at 1,024 buckets for the sort, limiting balance at high rank counts, and all seeds are held in memory with no out-of-core fallback. Exact validation at human scale needs an independent single-node FastGA reference, whose cost grows rapidly with the number of haplotypes. Agreement between runs at different rank counts is weaker evidence than agreement with FastGA, because it shares any error of our own pipeline.

Acknowledgements.

Author contributions.

Supplementary data. Supplementary data are available at *Bioinformatics* online.

Conflict of interest. None declared.

Funding. This work was supported by the Full Agency Name [grant number xxxx].

Data availability.

References

[1] Myers, G., Durbin, R. & Zhou, C. FastGA: fast genome alignment. *Bioinformatics Advances* **5**, vbaf238 (2025).

[2] Garrison, E. FastGA for pangenome self-alignment. Unpublished workflow document (2025).

[3] Guarracino, A. Pangenomes of multiple species for the "building pangenome graphs" publication. Zenodo (2023).

[4] Diwan, S. DistributedPanGA: A distributed-memory pangenome self-alignment pipeline. https://github.com/SakshamDiwan/DistributedPanGA (2026).

[5] Guidi, G. *et al. Distributed-memory parallel contig generation for de novo long-read genome assembly. Proceedings of the 51st International Conference on Parallel Processing (ICPP '22)*, 1–11 (ACM, 2022).